Churn Prediction Pipelines — Retention Architect (HackNU 2026)

Task: Two-stage classifier separating `not_churned` / `vol_churn` / `invol_churn`
Evaluation: PR-AUC, F1-score, Actionable Insights, Explainability
Data grain: One row per `user_id` (~90k train rows, ~141 features)

Architecture Overview

All pipelines share the same two-stage cascade structure. Only the model choices, hyperparameter grids, and imbalance strategies differ.

```
All users
    |
    ▼
┌──────────────────────────────────────┐
|  Stage 1: Churned vs Not Churned    |  ← trained on full dataset
|  Features: G, PU, Q, CS2, CS3      |
└──────────────────────────────────────┘
    |                    |
    ▼                    ▼
Not churned         Predicted churned
(P(churn) score)         |
                         ▼
                  ┌──────────────────┐
                  |  Zero-gen gate   |  ← hard rules before Stage 2
                  |  (X9, T_NEW1)    |
                  └──────────────────┘
                         |
                         ▼
                ┌──────────────────────────┐
                |  Stage 2: Vol vs Invol Churn  |  ← trained on churned subset
only
                |  Features: T, T_NEW, X1/X2/X7|
                └──────────────────────────┘
                    |              |
                    ▼              ▼
               Voluntary        Involuntary
             (discount offer)  (retry logic)
```

Zero-generation hard-rule gate

Before any user reaches Stage 2, apply this deterministic filter:

```python
```

```python
def apply_zero_gen_gate(df):
    # Free-tier users: not churned, exclude entirely
    df.loc[df['is_likely_free_tier_user'] == 1, 'final_label'] = 'not_churned'

    # Failed payment, zero successful: definitionally involuntary
    df.loc[
        (df['has_failed_but_no_successful_payment'] == 1) &
        (df['is_likely_free_tier_user'] == 0),
        'final_label'
    ] = 'invol_churn'

    # Remaining zero-gen users: flag and pass to Stage 2 with is_zero_gen=1
    df['is_zero_gen_user'] = (df['total_generations'] == 0).astype(int)

    return df
```

Cross-validation harness (shared by all pipelines)

```python
```

```python
import numpy as np
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import average_precision_score, f1_score

def run_two_stage_cv(build_s1, build_s2, X, y_binary, y_volInv, n_splits=5):
    """
    y_binary:  0=not_churned, 1=churned (vol or invol)
    y_volInv:  0=vol_churn, 1=invol_churn (only valid where y_binary==1)
    """
    cv = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)
    oof_s1 = np.zeros(len(X))
    oof_s2 = np.zeros(len(X))

    for fold, (train_idx, val_idx) in enumerate(cv.split(X, y_binary)):
        X_tr, X_val = X.iloc[train_idx], X.iloc[val_idx]
        y_tr, y_val = y_binary[train_idx], y_binary[val_idx]

        # Stage 1
        s1 = build_s1()
        s1.fit(X_tr, y_tr)
        oof_s1[val_idx] = s1.predict_proba(X_val)[:, 1]

        # Stage 2: train only on churned subset of training fold
        churn_tr  = train_idx[y_binary[train_idx] == 1]
        churn_val = val_idx[y_binary[val_idx] == 1]

        if len(churn_tr) > 0 and len(churn_val) > 0:
            s2 = build_s2()
            s2.fit(X.iloc[churn_tr][T_FEATURES], y_volInv[churn_tr])
            oof_s2[churn_val] = s2.predict_proba(
                X.iloc[churn_val][T_FEATURES]
            )[:, 1]

        print(f"Fold {fold+1} — S1 PR-AUC: "
              f"{average_precision_score(y_val, oof_s1[val_idx]):.4f}")

    return oof_s1, oof_s2


# Feature routing
S1_FEATURES = G_FEATURES + PU_FEATURES + Q_FEATURES + [
    'cs2_engagement_health', 'cs3_commitment_score'
]
T_FEATURES = T_FEATURES + T_NEW_FEATURES + [
    'x1_payment_failure_timing', 'x2_active_during_failure',
    'x7_time_to_first_payment_issue', 'x12_failed_before_success',
```

```
    'x13_failed_txn_to_gen_gap', 'cs1_payment_resilience'
]
```

Threshold tuning (shared by all pipelines)

Never use the default 0.5 threshold. Tune on OOF predictions:

```python
from sklearn.metrics import precision_recall_curve

def best_f1_threshold(y_true, y_prob):
    precision, recall, thresholds = precision_recall_curve(y_true, y_prob)
    f1_scores = 2 * precision * recall / (precision + recall + 1e-9)
    best_idx  = np.argmax(f1_scores)
    return thresholds[best_idx], f1_scores[best_idx]

threshold_s1, best_f1 = best_f1_threshold(y_binary_val, oof_s1_val)
print(f"Optimal S1 threshold: {threshold_s1:.3f}, F1: {best_f1:.4f}")
```

Pipeline A — LightGBM + XGBoost

Recommended starting point. Fastest to implement, best SHAP support, strong PR-AUC.

Stage 1: LightGBM with focal loss

```python
```

```python
import lightgbm as lgb
import numpy as np

def focal_loss_objective(y_pred, dataset):
    """Custom focal loss for LightGBM. Start with gamma=2, alpha=0.25."""
    gamma = 2.0
    alpha = 0.25
    y_true = dataset.get_label()
    p      = 1.0 / (1.0 + np.exp(-y_pred))
    grad   = alpha * (1 - p)**gamma * (gamma * p * np.log(p + 1e-9) + p - y_true)
    hess   = alpha * (1 - p)**gamma * p * (1 - p)
    return grad, hess

def focal_loss_eval(y_pred, dataset):
    y_true = dataset.get_label()
    p      = 1.0 / (1.0 + np.exp(-y_pred))
    loss   = -np.mean(0.25 * (1-p)**2 * y_true * np.log(p + 1e-9) +
                      0.75 * p**2 * (1-y_true) * np.log(1-p + 1e-9))
    return 'focal_loss', loss, False

def build_lgbm_focal(params=None):
    default = dict(
        n_estimators=1000,
        learning_rate=0.05,
        num_leaves=95,
        min_child_samples=30,
        subsample=0.80,
        subsample_freq=1,
        colsample_bytree=0.70,
        reg_alpha=0.1,
        reg_lambda=1.0,
        n_jobs=-1,
        random_state=42,
    )
    if params:
        default.update(params)
    return lgb.LGBMClassifier(**default)

def fit_lgbm_focal(model, X_tr, y_tr, X_val, y_val):
    train_set = lgb.Dataset(X_tr, label=y_tr)
    val_set   = lgb.Dataset(X_val, label=y_val, reference=train_set)

    booster = lgb.train(
        params={**model.get_params(), 'objective': focal_loss_objective, 'verbose':
        train_set=train_set,
        valid_sets=[val_set],
```

```
        feval=focal_loss_eval,
        callbacks=[lgb.early_stopping(50), lgb.log_evaluation(100)],
    )
    return booster
```

Hyperparameter search grid — Optuna, 100 trials:

```python
import optuna

def objective_s1_A(trial):
    params = {
        'n_estimators':      trial.suggest_int('n_estimators', 800, 1500),
        'learning_rate':     trial.suggest_float('lr', 0.03, 0.07, log=True),
        'num_leaves':        trial.suggest_int('num_leaves', 63, 127),
        'min_child_samples': trial.suggest_int('min_child_samples', 20, 50),
        'subsample':         trial.suggest_float('subsample', 0.70, 0.85),
        'colsample_bytree':  trial.suggest_float('colsample', 0.60, 0.80),
        'reg_alpha':         trial.suggest_float('alpha', 0.0, 1.0),
        'reg_lambda':        trial.suggest_float('lambda', 0.5, 5.0),
        # Focal-specific (passed separately to lgb.train)
        'focal_gamma':       trial.suggest_float('gamma', 1.5, 2.5),
        'focal_alpha':       trial.suggest_float('f_alpha', 0.25, 0.50),
    }
    # Cross-validate and return mean OOF PR-AUC
    ...
    return pr_auc_mean
```

Stage 2: XGBoost with scale_pos_weight

```python
```

```python
from xgboost import XGBClassifier

def build_xgb_s2(y_volInv, params=None):
    neg = (y_volInv == 0).sum()
    pos = (y_volInv == 1).sum()
    spw = neg / pos  # automatic scale_pos_weight

    default = dict(
        n_estimators=700,
        max_depth=5,
        learning_rate=0.05,
        scale_pos_weight=spw,
        min_child_weight=10,
        gamma=0.1,
        subsample=0.80,
        colsample_bytree=0.75,
        reg_alpha=0.1,
        reg_lambda=2.0,
        eval_metric='aucpr',
        random_state=42,
        n_jobs=-1,
    )
    if params:
        default.update(params)
    return XGBClassifier(**default)
```

Hyperparameter search grid — Optuna, 60 trials:

```python
def objective_s2_A(trial):
    params = {
        'n_estimators':     trial.suggest_int('n_estimators', 500, 1000),
        'max_depth':        trial.suggest_int('max_depth', 4, 6),
        'learning_rate':    trial.suggest_float('lr', 0.03, 0.08, log=True),
        'min_child_weight': trial.suggest_int('mcw', 5, 15),
        'gamma':            trial.suggest_float('gamma', 0.0, 0.3),
        'subsample':        trial.suggest_float('subsample', 0.75, 0.90),
        'colsample_bytree': trial.suggest_float('colsample', 0.65, 0.85),
        'reg_alpha':        trial.suggest_float('alpha', 0.0, 1.0),
        'reg_lambda':       trial.suggest_float('lambda', 1.0, 5.0),
    }
    ...
    return pr_auc_mean
```

Imbalance strategy

Stage	Strategy	Notes
S1	Focal loss (γ=2, α=0.25)	Down-weights easy negatives in loss; tune γ and α
S2	`scale_pos_weight = neg/pos`	Computed from churned subset class ratio
Sampling	None required	Focal loss replaces SMOTE in S1

Explainability (SHAP)

```python
import shap

explainer_s1  = shap.TreeExplainer(lgbm_booster)
shap_values_s1 = explainer_s1.shap_values(X_val)

explainer_s2  = shap.TreeExplainer(xgb_model)
shap_values_s2 = explainer_s2.shap_values(X_val_churn[T_FEATURES])

def get_reason_codes(shap_vals, feature_names, top_n=3):
    abs_shap = np.abs(shap_vals)
    top_idx  = np.argsort(abs_shap)[::-1][:top_n]
    return [(feature_names[i], round(float(shap_vals[i]), 4)) for i in top_idx]
```

When to use: Default choice. Fast iteration, excellent SHAP, easy to debug.

Pipeline B — LightGBM + Logistic Regression

Best for calibrated probabilities. Stage 2 LogReg coefficients are directly readable by a PM — no SHAP required.

Stage 1: LightGBM with is_unbalance

```python
```

```python
def build_lgbm_unbalanced(params=None):
    default = dict(
        n_estimators=1000,
        learning_rate=0.05,
        num_leaves=63,
        min_child_samples=40,
        subsample=0.80,
        subsample_freq=1,
        colsample_bytree=0.70,
        is_unbalance=True,   # LightGBM auto-weights classes
        reg_alpha=0.1,
        reg_lambda=1.0,
        n_jobs=-1,
        random_state=42,
        verbose=-1,
    )
    if params:
        default.update(params)
    return lgb.LGBMClassifier(**default)
```

Hyperparameter search grid — Optuna, 80 trials:

```python
def objective_s1_B(trial):
    params = {
        'n_estimators':      trial.suggest_int('n_estimators', 600, 1200),
        'learning_rate':     trial.suggest_float('lr', 0.04, 0.08, log=True),
        'num_leaves':        trial.suggest_int('num_leaves', 31, 95),
        'min_child_samples': trial.suggest_int('mcs', 30, 60),
        'subsample':         trial.suggest_float('subsample', 0.75, 0.85),
        'colsample_bytree':  trial.suggest_float('colsample', 0.65, 0.80),
        'reg_alpha':         trial.suggest_float('alpha', 0.0, 0.5),
        'reg_lambda':        trial.suggest_float('lambda', 0.5, 3.0),
    }
    ...
    return pr_auc_mean
```

Stage 2: Logistic Regression with elastic-net

Feature scaling is **required** before Stage 2 — T-features span very different numerical ranges.

```python
```

```python
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline as SKPipeline

def build_logreg_s2(params=None):
    default_lr = dict(
        C=1.0,
        penalty='elasticnet',
        solver='saga',
        l1_ratio=0.5,
        class_weight='balanced',
        max_iter=500,
        random_state=42,
    )
    if params:
        default_lr.update(params)

    return SKPipeline([
        ('scaler', StandardScaler()),
        ('clf',    LogisticRegression(**default_lr)),
    ])
```

Hyperparameter search grid — Optuna, 30 trials:

```python
def objective_s2_B(trial):
    params = {
        'C':        trial.suggest_float('C', 0.01, 10.0, log=True),
        'l1_ratio': trial.suggest_float('l1_ratio', 0.0, 1.0),
    }
    ...
    return pr_auc_mean
```

Reading coefficients as business logic:

```python
```

```
lr_model = pipeline.named_steps['clf']
coef_df  = pd.DataFrame({
    'feature': T_FEATURES,
    'coef':    lr_model.coef_[0],
}).sort_values('coef', ascending=False)

# Positive coef → pushes toward invol_churn
# Negative coef → pushes toward vol_churn
print(coef_df.head(10))
```

Imbalance strategy

Stage	Strategy	Notes
S1	`is_unbalance=True`	LightGBM internal weight scaling
S2	`class_weight='balanced'`	sklearn auto-computes from churned subset
Optional	SMOTE-ENN on S1 folds	Apply inside CV loop only; never leak to val

```python
from imblearn.combine import SMOTEENN

def apply_smoteenn(X_tr, y_tr):
    sm = SMOTEENN(random_state=42, n_jobs=-1)
    return sm.fit_resample(X_tr, y_tr)
```

When to use: When you need trustworthy probability scores for downstream user ranking, or when the PM wants plain-English feature importance without SHAP.

Pipeline C — CatBoost + CatBoost

Best for raw categorical features. Pass `country_code`, `dominant_failure_code_encoded`, plan tier directly — no encoding step required.

Stage 1: CatBoost with auto class weights

```python
```

```python
from catboost import CatBoostClassifier, Pool

CAT_FEATURES_S1 = [
    'country_code_encoded',
    'subscription_plan_ordinal',
    'dominant_generation_type_encoded',
    'dominant_aspect_ratio_encoded',
    'acquisition_channel_intent',
    'usage_plan_encoded',
    'role_encoded',
    'first_feature_encoded',
]

def build_catboost_s1(params=None):
    default = dict(
        iterations=1000,
        learning_rate=0.05,
        depth=7,
        l2_leaf_reg=3.0,
        bagging_temperature=0.75,
        auto_class_weights='Balanced',
        eval_metric='PRAUC',
        random_seed=42,
        verbose=100,
        early_stopping_rounds=50,
    )
    if params:
        default.update(params)
    return CatBoostClassifier(**default)

def fit_catboost_s1(model, X_tr, y_tr, X_val, y_val):
    train_pool = Pool(X_tr, label=y_tr, cat_features=CAT_FEATURES_S1)
    val_pool   = Pool(X_val, label=y_val, cat_features=CAT_FEATURES_S1)
    model.fit(train_pool, eval_set=val_pool, use_best_model=True)
    return model
```

Hyperparameter search grid — Optuna, 80 trials:

```python
```

```python
def objective_s1_C(trial):
    params = {
        'iterations':          trial.suggest_int('iterations', 800, 1500),
        'learning_rate':       trial.suggest_float('lr', 0.03, 0.07, log=True),
        'depth':               trial.suggest_int('depth', 6, 10),
        'l2_leaf_reg':         trial.suggest_float('l2', 1.0, 10.0),
        'bagging_temperature': trial.suggest_float('bagging_temp', 0.5, 1.0),
    }
    # auto_class_weights='Balanced' is fixed — do not tune
    ...
    return pr_auc_mean
```

Stage 2: CatBoost with manual class weights

```python
CAT_FEATURES_S2 = [
    'dominant_failure_code_encoded',
    'card_funding_type_encoded',
    'country_code_encoded',
]

def build_catboost_s2(y_volInv, params=None):
    neg = (y_volInv == 0).sum()
    pos = (y_volInv == 1).sum()

    default = dict(
        iterations=600,
        learning_rate=0.06,
        depth=6,
        l2_leaf_reg=5.0,
        class_weights={0: 1.0, 1: neg / pos},
        eval_metric='PRAUC',
        random_seed=42,
        verbose=50,
        early_stopping_rounds=40,
    )
    if params:
        default.update(params)
    return CatBoostClassifier(**default)
```

Hyperparameter search grid — Optuna, 50 trials:

```python
```

```python
def objective_s2_C(trial):
    params = {
        'iterations':    trial.suggest_int('iterations', 400, 800),
        'learning_rate': trial.suggest_float('lr', 0.04, 0.09, log=True),
        'depth':         trial.suggest_int('depth', 4, 7),
        'l2_leaf_reg':   trial.suggest_float('l2', 3.0, 10.0),
    }
    ...
    return pr_auc_mean
```

Imbalance strategy

Stage	Strategy	Notes
S1	`auto_class_weights='Balanced'`	CatBoost internal; equivalent to sklearn balanced
S2	`class_weights={0: 1, 1: neg/pos}`	Computed from churned subset at build time
Sampling	Not recommended	Ordered boosting already regularises small datasets

SHAP with CatBoost

```python
shap_values = catboost_model.get_feature_importance(
    Pool(X_val, cat_features=CAT_FEATURES_S1),
    type='ShapValues'
)
# Shape: (n_samples, n_features + 1) — last column is bias, drop it
shap_matrix = shap_values[:, :-1]
```

When to use: When you want to skip the encoding step entirely, or if high-cardinality categoricals like `bank_name` are kept in the feature set.

Pipeline D — Stacking Ensemble

Highest PR-AUC ceiling. Use when you have 4+ hours of compute and want to squeeze the last percentage points out. Most complex to implement.

Stage 1: StackingClassifier with meta-LogReg

Pre-tune each base learner independently (using Pipelines A–C) before building the stacker. Only tune the meta-learner's `C` in the stacking phase.

```python
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.calibration import CalibratedClassifierCV

def build_stacking_s1(scale_pos_weight):
    base_lgbm = lgb.LGBMClassifier(
        n_estimators=1000, learning_rate=0.05, num_leaves=95,
        is_unbalance=True, subsample=0.80, colsample_bytree=0.70,
        n_jobs=-1, random_state=42, verbose=-1,
    )
    base_xgb = XGBClassifier(
        n_estimators=700, max_depth=5, learning_rate=0.05,
        scale_pos_weight=scale_pos_weight,
        eval_metric='aucpr', n_jobs=-1, random_state=42,
    )
    base_cat = CatBoostClassifier(
        iterations=800, learning_rate=0.05, depth=7,
        auto_class_weights='Balanced', verbose=0, random_seed=42,
    )

    meta = LogisticRegression(C=1.0, max_iter=300, random_state=42)

    stacker = StackingClassifier(
        estimators=[
            ('lgbm', base_lgbm),
            ('xgb',  base_xgb),
            ('cat',  base_cat),
        ],
        final_estimator=meta,
        cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
        stack_method='predict_proba',
        passthrough=True,  # meta-learner also sees original features
        n_jobs=-1,
    )
    # Calibrate final probabilities
    return CalibratedClassifierCV(stacker, method='isotonic', cv='prefit')
```

Tuning — only the meta-learner, Optuna, 30 trials:

```python
```

```
def objective_meta(trial):
    params = {
        'final_estimator__C': trial.suggest_float('C', 0.01, 10.0, log=True),
        'passthrough':        trial.suggest_categorical('passthrough', [True, False]
    }
    ...
    return pr_auc_mean
```

Stage 2: Soft VotingClassifier

```python
from sklearn.ensemble import VotingClassifier

def build_voting_s2(y_volInv):
    neg = (y_volInv == 0).sum()
    pos = (y_volInv == 1).sum()
    spw = neg / pos

    xgb_s2 = XGBClassifier(
        n_estimators=700, max_depth=5, learning_rate=0.05,
        scale_pos_weight=spw, eval_metric='aucpr',
        n_jobs=-1, random_state=42,
    )
    cat_s2 = CatBoostClassifier(
        iterations=600, depth=6, learning_rate=0.06,
        class_weights={0: 1, 1: spw}, verbose=0, random_seed=42,
    )
    lr_s2 = SKPipeline([
        ('scaler', StandardScaler()),
        ('clf', LogisticRegression(
            C=0.1, penalty='elasticnet', solver='saga',
            l1_ratio=0.5, class_weight='balanced', max_iter=500,
        )),
    ])

    return VotingClassifier(
        estimators=[('xgb', xgb_s2), ('cat', cat_s2), ('lr', lr_s2)],
        voting='soft',
        weights=[2, 2, 1],   # tune: tree models typically outperform LR here
        n_jobs=-1,
    )
```

Tuning ensemble weights — Optuna, 20 trials:

```python
def objective_weights(trial):
    w_xgb = trial.suggest_int('w_xgb', 1, 4)
    w_cat = trial.suggest_int('w_cat', 1, 4)
    w_lr  = trial.suggest_int('w_lr',  1, 2)
    voter = VotingClassifier(..., weights=[w_xgb, w_cat, w_lr], voting='soft')
    ...
    return pr_auc_mean
```

Imbalance strategy

Stage	Strategy	Notes
S1	Per-base-learner (see A–C)	Each base handles its own imbalance
S2	Per-base-learner (XGB: spw, Cat: class_weights, LR: balanced)	Independently tuned
Meta	No sampling	Meta-learner sees calibrated OOF probs, not raw counts

Explainability for the ensemble

SHAP does not work directly on `StackingClassifier`. Use one of:

```python
# Option 1 (recommended for judges): SHAP on strongest base learner
explainer   = shap.TreeExplainer(stacker.estimators_[0])   # LightGBM
shap_values = explainer.shap_values(X_val)

# Option 2: interpret how much each base model's vote drove the final prediction
meta_input = stacker.transform(X_val)   # OOF-like features + optional original feat
meta_exp   = shap.LinearExplainer(stacker.final_estimator_, meta_input)
meta_shap  = meta_exp.shap_values(meta_input)
```

When to use: Final submission when time permits. Always implement and validate Pipeline A first.

Experiment Runner

```python
```

```python
import pandas as pd
from datetime import datetime

PIPELINE_REGISTRY = {
    'A': {'s1': build_lgbm_focal,      's2': build_xgb_s2},
    'B': {'s1': build_lgbm_unbalanced, 's2': build_logreg_s2},
    'C': {'s1': build_catboost_s1,     's2': build_catboost_s2},
    'D': {'s1': build_stacking_s1,     's2': build_voting_s2},
}

results = []

for name, pipe in PIPELINE_REGISTRY.items():
    print(f"\n=== Pipeline {name} ===")
    oof_s1, oof_s2 = run_two_stage_cv(
        pipe['s1'], pipe['s2'],
        X_train[S1_FEATURES], y_binary, y_volInv
    )

    churn_mask = y_binary == 1
    pr_auc_s1  = average_precision_score(y_binary, oof_s1)
    pr_auc_s2  = average_precision_score(y_volInv[churn_mask], oof_s2[churn_mask])
    thr, f1    = best_f1_threshold(y_binary, oof_s1)

    results.append({
        'pipeline':           name,
        'pr_auc_s1':          round(pr_auc_s1, 4),
        'pr_auc_s2':          round(pr_auc_s2, 4),
        'best_f1_s1':         round(f1, 4),
        'best_threshold_s1':  round(thr, 3),
        'timestamp':          datetime.now().isoformat(),
    })
    print(pd.DataFrame(results).tail(1).to_string(index=False))

results_df = pd.DataFrame(results).sort_values('pr_auc_s1', ascending=False)
print("\n=== Final Rankings ===")
print(results_df.to_string(index=False))
```

Full Inference Pipeline

```python
```

```python
def predict_churn(user_df, s1_model, s2_model, threshold_s1=0.30):
    """Two-stage inference for a batch of users."""

    user_df = apply_zero_gen_gate(user_df)
    needs_model = user_df['final_label'].isna()

    # Stage 1
    probs_s1 = s1_model.predict_proba(user_df.loc[needs_model, S1_FEATURES])[:, 1]
    user_df.loc[needs_model, 'churn_probability'] = probs_s1
    user_df.loc[needs_model & (probs_s1 < threshold_s1), 'final_label'] = 'not_churn

    # Stage 2 on predicted churned
    churn_idx = needs_model & (user_df['churn_probability'] >= threshold_s1)
    if churn_idx.sum() > 0:
        probs_s2 = s2_model.predict_proba(user_df.loc[churn_idx, T_FEATURES])[:, 1]
        user_df.loc[churn_idx, 'invol_churn_probability'] = probs_s2
        user_df.loc[churn_idx, 'final_label'] = np.where(
            probs_s2 >= 0.5, 'invol_churn', 'vol_churn'
        )

    return user_df[['user_id', 'final_label',
                    'churn_probability', 'invol_churn_probability']]
```

Submission Strategy

Scenario	Recommended approach
< 2 hours remaining	Pipeline A with default hyperparameters
2–4 hours remaining	Pipeline A with Optuna (100 S1 trials, 60 S2 trials)
4+ hours remaining	Pipeline D (S1 stacking) + Pipeline A (S2 XGBoost)
Explainability slide	Always use SHAP from LightGBM or XGBoost, not the ensemble
PM strategy deck	Use Pipeline B Stage 2 LogReg coefficients — readable as plain English

---

Dependencies

```txt
```

```
lightgbm>=4.0
xgboost>=2.0
catboost>=1.2
scikit-learn>=1.3
imbalanced-learn>=0.11
shap>=0.44
optuna>=3.4
pandas>=2.0
numpy>=1.24
```

bash

```bash
pip install lightgbm xgboost catboost scikit-learn imbalanced-learn shap optuna pand
```